

00:00

Right. Hi everyone. This is our lecture week 9, and in this lecture we're going to talk about making predictions, specifically a specific type of regression: simple regression. The objectives are to use the correlation method, scatter plots, and heat maps to identify correlations; distinguish between simple regression and multiple regression; use scikit-learn to create and validate simple linear regression models; use the models we create to make predictions; and use Seaborn to plot predicted values and residuals. We will also use Seaborn's `lplot` function to create and plot linear, logistic, polynomial, and LOWESS regression models, as well as the `residplot` function to plot residuals of a linear regression model.

02:06

Okay, so without further ado, we'll first talk about correlation. We are interested to know what correlation is and how it is defined between two variables. To define correlation, we need a measure. One of the main measures of correlation is the correlation coefficient, also referred to as the *r*-value. In statistics, it is used to measure the strength of the correlation between two variables, and it represents the strength and direction of a linear relationship between variables. We highlight the term linear because the correlation coefficient is only able to capture linear relationships. If the correlation between two variables is zero, that doesn't mean there is no relationship between them; it means there is no linear relationship between them. This is an important thing to keep in mind.

03:33

Another important term is direction. The correlation coefficient captures the direction of the relationship in addition to its strength. If one variable increases, does the other one decrease or increase? The measure is referred to as the *R*-value. We use ρ to represent it: ρ of *x* and *y* equals the covariance of *x* and *y* divided by the standard deviation of *X* times the standard deviation of *Y*. *X* and *Y* are the two variables whose relationship we want to find. They could be, for instance, someone's height and weight. The covariance of *x* and *y*, divided by the standard deviation of *X* times the standard deviation of *Y*, gives us the *R*-value for variables *X* and *Y*.

04:50

In the graph, the relationship between two variables is shown with the x variable on the x-axis and the y variable on the y-axis. In the first graph there is a positive relationship between x and y. As we move along the x-axis, the value of y is increasing in general. It is not the case for every single point, but in general the value is increasing, so we can say there is a positive correlation between x and y. In the second graph there is a negative relationship: as we move along the x-axis, the value of y is decreasing. The slope of the line would be negative, whereas in the first graph the slope is positive. In the third graph there is almost no linear relationship between x and y. Again, this does not mean there is no relationship between X and Y; there is no linear relationship, so the correlation coefficient and the slope are almost zero.

06:54

As an example, let's say you want to identify the relationship between the height and weight of someone or a group of individuals. You measure their heights and weights and calculate the correlation coefficient. A positive correlation coefficient, or r-value, close to 1 indicates a strong positive relationship, meaning that as height increases, weight tends to increase as well. A negative r-value close to minus 1 indicates a strong negative relationship, meaning that as height increases, weight tends to decrease. If the r-value approaches 0, there is no linear relationship between height and weight.

08:14

Here you can see graphs with different values for R. An R-value of 1 indicates perfect positive correlation. A smaller positive value still indicates a positive correlation, but it is not perfect. It can be a strong or high positive correlation, or a weak or low positive correlation. Eventually there is no linear relationship, which is zero correlation. As we move through the remaining graphs, the relationship becomes negative: first a weak negative correlation, then a stronger negative correlation, and eventually a perfect negative correlation with a value of minus 1. The r-value varies between one and minus one, and it can take any value in that range, including both endpoints.

09:49

To calculate the r-value, we can use the COR function of a data frame. Let's look at an example. We create a data frame from data composed of three variables, with each

variable having its own values. We then calculate the correlation metrics, which comprise the r-values for pairs of variables. The COR function creates the correlation matrix, and the print function prints the matrix. The matrix has three rows and three columns corresponding to the three variables. It shows the r-values that capture the relationship between pairs of variables. Variable 1 against variable 1 represents the relationship between a variable and itself, which is a perfect correlation, so its value is 1. We are not interested in calculating this because it is already known.

11:55

The R-value is calculated by dividing the covariance of X and Y by the multiplication of the standard deviation of X and the standard deviation of Y. Sigma represents the standard deviation for x and y. When x and y are the same, the covariance of x and y is the variance of x, and the denominator also becomes the variance of x. Therefore, variance of x divided by variance of x equals 1. That is the perfect correlation between a variable and itself. For the other variables, the r-value is calculated between variable 1 and variable 2, variable 1 and variable 3, and variable 2 and variable 3. The reverse pairs are also present, but they are redundant because the covariance of X and Y equals the covariance of Y and X. Therefore, the R-value for X and Y is the same as the R-value for Y and X.

15:53

Because the order does not matter, we are selecting combinations rather than permutations. With three variables, selecting two variables gives three combinations: three factorial divided by two factorial times one factorial, which equals three. The three unique values are minus 0.45, 0.68, and 0.07. Each is repeated because when we calculate the value for x and y, we also get the value for y and x. The value calculated for the correlation between variable one and variable two is the same as the value calculated between variable two and variable one.

17:49

The data set we are going to use in the coming slides is the housing data set. This data set includes housing or property data for properties whose price is less than one million, greater than zero, and whose living area is less than 8,000 square feet. We filter the data set based on price and living area. We also apply a lambda function to check whether the basement square footage is greater than zero. If it is greater than zero,

ro, the value is set to true; otherwise it is set to false. This creates a new column indicating whether the property has a basement. We drop columns that we are not going to use, such as location and city, and obtain a data frame containing the fields needed for the analysis.

19:52

Using the info function, we get the structure of the data and an overview of the data frame. We want to identify correlations between pairs of variables using R-values or correlation coefficient values, so we use the COR function. We call the function on the data frame to generate the correlation metrics. If we print the head of the correlation matrix, we are showing only the first five rows, not the whole matrix. There are eleven variables, so the matrix is 11 by 11. The matrix contains the r-values. For example, the r-value calculated for price and square feet living is 0.607379, while the value for square feet living and square feet above is 0.848312.

22:14

Another way to extract r-values is to select the price column from the correlation matrix. This lets us compare price against every other variable, including square feet living, square feet above, and everything else. The values are sorted by price in descending order, using ascending equals false. This makes it easy to identify which variable has the strongest relationship or correlation with price. Square feet living has the strongest relationship, and its value is higher than the other values. It is meaningless to compare a variable against itself, because the relationship of a variable with itself is always perfect. As we move down the list, the correlations between the variables and price become weaker, with condition at the bottom.

24:35

There is an important question: correlation is not causation. If two variables are strongly correlated, that means when x increases, y also increases, but it does not mean there is causation between them. It does not mean that increasing x causes increasing y. This is correlation, not causation. So can we say that a higher value of square feet living leads to a higher price? Try to answer that question based on what I just explained.

02:32

We can identify correlations using scatter plots. Until now we have learned how to identify correlations using R-values and correlation metrics. Now we want to identify correlations by looking at a plot, specifically a scatter plot. We create a plot of the data comparing the values of X and Y. We use the Seaborn library and the `sns.replot` function. The data is the housing data, which includes all fields and columns, but we want to show the relationship between square feet living and price. By looking at the resulting plot, we can see a positive relationship: as we increase square feet living along the x-axis, price generally increases. It does not increase all the time, but the general trend is upward, which explains the positive R-value.

27:07

The plot visualizes the relationship between square feet living and price, and from the scatter plot we can guess the relationship. In another plot we compare year built and price. There is no meaningful relationship between them. The data is spread all over the X and Y axes, so we cannot say that increasing one variable increases or decreases the other. We cannot draw a conclusion about a positive or negative correlation between those two variables. We can print the correlation metrics for price, square feet living, and square feet above, and use `pairplot` to create plots of the relationships between the variables. The off-diagonal plots show scatter plots for the different pairs, while the diagonal shows the distribution of each variable because a variable compared with itself would have a perfect relationship.

29:40

On the diagonal, the variables' distributions are displayed instead of their perfect self-correlations. Elsewhere, we can see the relationships between the variables. In general, the relationships are positive. There is a strong positive relationship between square feet living and square feet above. The same relationship appears in the opposite order because the pair is repeated. There are also positive relationships between other variables, but they are not as strong as the correlation between square feet living and square feet above. We can also use a KDE on the diagonal instead of a histogram. By setting `diagonal kind` to `KDE`, Seaborn shows the distribution of the variables using a KDE function. The `diagonal kind` can be changed to different values.

32:33

Instead of a scatter plot, we can use a heat map to show relationships between variables.

bles. The diagonal contains perfect relationships between each variable and itself, so we are not interested in those values. Darker blue colors show stronger relationships, and the color bar helps us interpret the map. A value of 1 shows the strongest positive relationship, while minus 1 shows the strongest negative relationship. As the colors move toward white, the relationship becomes more negative; the range goes from white to dark blue, almost black. Zero lies in the middle. Values below zero produce brighter colors, and values above zero produce darker colors. In this heat map, there is a strong positive relationship between square feet above and square feet living, and also between price and square feet living.

34:56

One issue with a heat map is that colors provide an initial overview but can be confusing because it may not be clear exactly where values fall, and some colors may be difficult to distinguish. A more readable approach is to show the numerical values together with the colors. We can select price and show its relationship with every other variable, while displaying the r-values. We format the r-values to two decimal points, so 1 becomes 1.00 and values such as 0.607379 become 0.61. The values are rounded to two decimal points and displayed in the heat map. This is another way to use heat maps.

36:47

Now let's talk about predictive models. We have discussed correlation, and now we want to talk about prediction and predictive models. The purpose of prediction is to use data to predict future values. So far we have covered understanding data, cleaning data, preparing data, and analyzing data. We may be interested in creating predictive models so that we can predict future values. Predictive models are statistical or machine-learning models that use data to make predictions. They identify patterns in the data and use those patterns to predict outcomes. They can be used in customer relationship management, fraud detection, risk assessment, marketing, healthcare, business, and organizations to improve decision-making and optimize performance.

38:12

To build a predictive model, we still need to collect data. We collect data on variables expected to influence the outcome. We may be interested in a subset of the data, so we need at least the variables required to make the prediction. We have an outcome,

which is what we are going to predict, and we need data to make a prediction based on those variables. That is the data collection stage. Once the data is collected, we may need to clean and preprocess it. Preprocessing includes cleaning, preparing, and transforming the data into a format that can be used to build the model and make predictions.

03:58

The third step is to select a model. We choose an appropriate model based on the nature of the data and the problem being addressed. This is a critical step: we need to find a model that works best with our data. Sometimes there are best practices, but sometimes we need to try different models and see which one works better and makes better predictions. A lot of experience can be involved in choosing a good or appropriate model. The fourth step is to train the model. Once we select the model, we use the data we collected to train it. Training means using the data so that the model can make predictions based on its history and tell us what the outcome may be. For example, using the history of weather in a particular area and certain parameters, a model can predict whether it will rain.

41:50

Once the model has been created and trained, we evaluate it using different measures to make sure its predictions are accurate enough. Then we deploy the model and use it in practice to make actual predictions. Predictive models include many types of machine-learning algorithms: regression models, including simple and multiple regression and linear and nonlinear regression; classification models; clustering models; time-series models; and neural-network models. The appropriate choice depends on the data, the problem, the outcome we want to predict, and the kind of data we are dealing with, such as text or images.

43:28

In this lecture we focus on linear regression models. We may also cover some nonlinear models, but the main focus is simple linear regression. Linear regression assumes that the relationship between variables is linear. There is a dependent variable whose value we want to predict, and that value can be expressed as a linear combination of one or more independent variables. The goal of linear regression is to find the coefficients of the independent variables that best fit the data, so the model can be used to

o make predictions on new data. With one independent variable, we have simple linear regression. With more than one independent variable, we have multiple linear regression or multiple regression.

04:38

Here are examples of linear regression. We might predict revenue, the dependent variable, based on an advertising budget, the independent variable. DV stands for dependent variable, or what we want to predict, and IV stands for independent variable. This is simple regression because there is only one independent variable. Another example is multiple regression, where the dependent variable is a patient readmission rate and the independent variables include hemoglobin count, prior admission, blood pressure, current medications, and age. The other examples are also multiple regression examples. They are assumed to be linear regression because we assume the dependent variable can be predicted from a linear combination of the independent variables.

48:35

To work with linear regression models, we can use scikit-learn, also called sklearn. It is a popular machine-learning library in Python that provides tools for building predictive models, including classification, regression, and clustering. Before working with scikit-learn, we need to understand `train_test_split`. This utility function, from `sklearn.model_selection`, divides a data set into two subsets: a training set and a testing set. We use the training set to train the regression model and the testing set to test or evaluate it. The function randomly shuffles the data, assigns a specified percentage to the training set, and assigns the remaining percentage to the testing set. The testing data is unseen data, meaning it has not yet been provided to the model.

51:42

The scikit-learn library also has a `LinearRegression` class. It is an implementation of linear regression and is used to model the relationship between a dependent variable and one or more independent variables. We import `train_test_split` from `sklearn.model_selection` and `LinearRegression` from `sklearn.linear_model`. The train-test-split function partitions the data into a training subset and a testing subset, and the `LinearRegression` class creates the regression model. To create a model, we use the train-test-split function on the housing data frame, selecting square feet living as the feature and

d price as the target. Price is the variable we want to predict, based on square feet living. This is clearly simple regression because there is only one independent variable. The independent variable is also called the feature, and the dependent variable is called the target.

53:28

We set `test_size` to 0.33, so 33% of the records in the original data frame are allocated to testing and the rest to training. We use most of the records to train the regression model and 33% to test it. The function returns four arrays: `X train`, `X test`, `Y train`, and `Y test`. `X` represents the independent variable, which here is square feet living, and `Y` represents price. We create a linear regression model by calling the constructor of the `LinearRegression` class, assigning the resulting instance to `linear_model`. Then we train the model using the `fit` function and the training data, both the independent and dependent variables. In other words, we tell the model that for different values of square feet living, it should learn the corresponding price.

57:49

When creating the split, we set a random seed. In machine learning, this makes the result reproducible. When the `random_state` parameter in scikit-learn's `train_test_split` function is fixed, the split is deterministic and the same result is achieved every time the line is executed. The benefit is that we get the same training subset and the same testing subset each time. To validate a linear regression model, we can use the R-squared score. This is the square of the r-value; in this example it is about 0.35. We obtain it by calling the `score` function of the linear model with the independent variable from the test set and the dependent variable from the test set.

01:00:28

The R-squared score is also called the coefficient of determination. It measures the proportion of variance in the dependent variable that is explained by the independent variable or variables in a linear regression model. In other words, it shows how well the independent variable can predict variation in the dependent variable. The R-squared score ranges from 0 to 1, with higher values indicating a better fit to the data. A value of 0 indicates that the model explains none of the variability of the response data around its mean, while a value closer to 1 indicates that the model explains more or all of the variability. By response data, we mean the predicted value or dependent variable.

variable.

01:02:09

It is important to note that a higher R-squared value does not necessarily mean the model is a good fit. R-squared is one criterion and does not guarantee that the model fits the data well. Other factors include overfitting and outliers. Overfitting occurs when the model is too precise: it gives very high accuracy for the training data set but low accuracy for the test data set. It fits the training set very well but does not do a good job predicting the dependent variable in the test set. It is also important to consider the context and field of study, because different fields may use different standards for measuring model performance. It is often recommended to use R-squared together with other statistical measures or visualizations.

01:04:38

We have learned how to create a linear regression model and use the R-squared score to evaluate it, but R-squared by itself cannot tell us whether a regression model is a good fit. We can use the predict function or method of the linear model to make predictions. The model is an instance of the LinearRegression class. We pass X test to linear_model.predict. X train, X test, Y train, and Y test are the arrays returned by train_test_split. We provide the independent variable from the testing data set as input to predict the dependent variable. Here, X represents square feet living and Y represents price, so we pass the square feet living values to predict prices. The method returns an array of predicted values.

01:07:30

For displaying the results, we concatenate square feet living, price, and predicted price. The actual price in the testing data was not provided to the model; the model was trained only with the training data. We then ask it to predict the prices for the test data. The predicted values can differ from the actual values. For example, one predicted price may be lower than the actual price, while another may be significantly higher. Some predictions are close to the actual values, but the overall accuracy does not seem very good. The R-squared score is 0.35, meaning that the model explains about 35% of the variation.

01:10:49

We can plot the predicted data using Seaborn's replot function. We create a data frame from the predicted prices and label the column price_predicted. We join X test, which is square feet living, and Y test, which is price, from the testing data set. Then we melt the data so that price and price_predicted become values represented by price_type and price_value. For each square feet living value, we have an actual price and a predicted price. Melting the data makes it easier to visualize. We use sns.replot, set square feet living on the x-axis, price_value on the y-axis, and use hue to distinguish actual price from predicted price. The predicted prices are shown in orange and the actual prices in blue. The variation shows that the model captures some of the variance but cannot predict most points precisely.

01:14:55

We can plot linear regression directly with Implot. This is a quick way to see the regression line and compare it with the actual prices. We pass the housing data, use square feet living as x and price as y, and the line can be used to make predictions for a given amount of living space. The values shown on the plot may be formatted rather than exact values. The predicted price may differ from the actual price in most cases. Comparing the regression line with the actual data points may suggest that square feet living is not a sufficient indicator of price by itself, and simple linear regression may not be the best model. We can start with a simple linear regression and then consider a more sophisticated model.

01:17:50

We can also plot residuals. A residual is the difference between the actual price and the predicted price. The combined data frame contains price and price_predicted, so we calculate a new residual column showing how far the prediction is from the actual value. Positive residuals occur where the actual price is greater than the predicted price, and negative residuals occur where the actual price is less than the predicted price. We can scatter-plot residuals against square feet living, with square feet living on the x-axis and residuals on the y-axis. A horizontal line highlights zero. Points above zero are positive residuals, and points below zero are negative residuals.

01:19:50

Here is a question about correlation versus regression: is there any relationship between the correlation coefficient and the slope of the regression line in simple linear re

gression? This is something to investigate. The relationship is interesting because the slope of the regression line can be calculated from the covariance of the dependent and independent variables and the variance of the independent variable. We also have videos that summarize regression and explain how to calculate linear regression.

01:20:58

There are different types of regression. We have simple and multiple regression, and linear and nonlinear regression. Logistic regression is one of the nonlinear types. It can also be simple or multiple depending on whether there is one independent variable or more than one. Logistic regression is a statistical method used to analyze data with a binary or categorical outcome. We may want to know whether the predicted variable is true or false, or 0 or 1. Logistic regression can be used for classification, for example to decide whether an image belongs to a human or not. The model estimates the probability of the outcome based on predictor variables, which could be image features.

01:22:33

We can use the same `lplot` function to plot a logistic regression model by setting the logistic parameter to true. The function estimates the value of house basement based on price, showing whether a house has a basement based on its price. The regression line shows the probability that a house has a basement. As price increases, the chance of having a basement increases. `HasBasement` takes only two values, 0 and 1, or true and false. A value of 1 means the property has a basement, while 0 means it does not. We can set the confidence interval to 95%, creating a shaded area around the regression line that represents the interval containing the line with 95% confidence.

01:24:42

Another type of regression is polynomial regression. Polynomial regression models the relationship between a dependent variable, y , and one or more independent variables, x or x -values such as x_1 , x_2 , and x_3 . With one variable it can be simple polynomial regression; with more than one it can be multiple. The polynomial equation can take quadratic, cubic, or higher-order forms. Polynomial regression can be used when the relationship between the independent and dependent variables is not linear and cannot be adequately captured by a simple linear regression model. If the correlation

coefficient is close to zero, that does not mean there is no relationship; it means there is no linear relationship, so a polynomial model may be more appropriate.

01:26:31

To plot polynomial regression, we provide the data, set x to year and y to death rate, and use the mortality data set rather than the housing data set. Year is the independent variable and death rate is the dependent variable. We plot different age groups and set the confidence interval to none. We set the number of x -bins to 20, dividing the x -axis into 20 evenly spaced bins, and set the order to 3, creating a cubic polynomial regression. The resulting cubic regression line attempts to predict death rates based on year. We can also plot residuals for a polynomial regression using Seaborn's `residplot` function, passing the data frame, x , y , and order equal to 3. The function plots the differences between predicted and actual death rates.

08:51

The residuals for the polynomial regression are close to zero after about 1950. At the beginning, there are positive and negative values, with a larger gap between estimated and actual values. Later, the residuals approach zero. There is another video about polynomial regression that summarizes these ideas and provides more information.

01:30:27

There is one more type of regression: LOWESS regression, or locally weighted scatter plot smoothing. It is a nonparametric regression method used to fit a smooth curve to a scatter plot of data points. Unlike parametric regression methods such as linear regression, which assume a specific functional form for the relationship between dependent and independent variables, LOWESS makes no assumption about the function form. It does not assume that the relationship is linear, quadratic, cubic, or another particular type. Instead, it uses a weighted average of nearby data points to estimate the curve. It considers a window, moves the window across the data along the x -axis, and finds curves that fit the data points in that window. The weights are based on the distance between the point of interest and other points in its neighborhood.

01:32:49

LOWESS regression is particularly useful when the relationship between variables is complex or nonlinear. We can plot it using `lplot`. X is the variable used to make the

prediction, and y is what we predict, such as death rate. We set `lowess` to `true`, which creates and plots a LOWESS regression for different age groups. Because `hue` is set to age group, different regression lines are created for the different groups. This is the result of using LOWESS instead of polynomial or linear regression. It makes no assumption about the shape of the function; values are calculated in each moving window to create a curve that fits the data points.

01:34:42

We are not going into the details of exactly how LOWESS regression works or its pros and cons. In this lecture we wanted to briefly cover simple regression and different types of simple regression: linear regression, logistic regression, polynomial regression, and LOWESS regression. All these regression models can also be multiple regression models. In this lecture we focused on simple regression, meaning that we considered only one independent value or one independent variable to predict the dependent variable. We covered different regression models that could be either simple or multiple, and showed examples of simple regression for each model. Thanks for your attention. I'm going to close the lecture.